

# ML Assignment 4

Sanjitha S (2023CS50734), Nishant Ranjan (2023CS10607)

November 2025

## 1 RNN

### 1.1 Theory of RNN and Bahdanau Attention

#### 1.1.1 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by maintaining an internal hidden state that captures information from previous time steps.

For our maze-solving task, we employ an encoder-decoder architecture:

**Encoder:** The encoder processes the input sequence  $X = (x_1, x_2, \dots, x_{T_x})$  token-by-token, where each  $x_i$  represents a token in the maze description (adjacency list, origin, and target coordinates). At each time step  $t$ , the encoder updates its hidden state:

$$h_t = f(x_t, h_{t-1}) \quad (1)$$

where  $f$  is a non-linear function ( $\tanh$  in our case). The final hidden state  $h_{T_x}$  serves as the context vector, compressing all information from the input sequence.

**Decoder:** The decoder generates the output sequence  $Y = (y_1, y_2, \dots, y_{T_y})$  representing the path through the maze. At each time step  $t$ , it produces:

$$s_t = g(y_{t-1}, s_{t-1}, c_t) \quad (2)$$

$$p(y_t | y_1, \dots, y_{t-1}, X) = \text{softmax}(W_s s_t) \quad (3)$$

where  $s_t$  is the decoder's hidden state,  $c_t$  is the context vector (enhanced by attention), and  $W_s$  is a learned weight matrix.

#### 1.1.2 Bahdanau Attention Mechanism

The traditional encoder-decoder architecture compresses the entire input sequence into a single fixed-length context vector, which can become a bottleneck for long sequences. The Bahdanau attention mechanism addresses this limitation by allowing the decoder to dynamically focus on different parts of the input sequence at each decoding step.

**Attention Computation:** At each decoder time step  $t$ , the attention mechanism computes a context vector  $c_t$  as a weighted sum of all encoder hidden states:

$$c_t = \sum_{i=1}^{T_x} \alpha_{ti} h_i \quad (4)$$

where  $\alpha_{ti}$  is the attention weight indicating the importance of encoder hidden state  $h_i$  when generating output  $y_t$ . These weights are computed as:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^{T_x} \exp(e_{tj})} \quad (5)$$

$$e_{ti} = a(s_{t-1}, h_i) \quad (6)$$

Here,  $e_{ti}$  is the alignment score computed by an alignment model  $a$ , which is a single-layer feedforward neural network:

$$e_{ti} = v_a^T \tanh(W_a s_{t-1} + U_a h_i) \quad (7)$$

where  $W_a$ ,  $U_a$ , and  $v_a$  are learned weight matrices. This alignment model jointly learns to align and translate, determining which parts of the input sequence are relevant for generating the current output token.

**Decoder with Attention:** Following Equation (3) of the Bahdanau paper, the decoder’s hidden state is computed as:

$$s_t = f(y_{t-1}, s_{t-1}, c_t) \quad (8)$$

where  $c_t$  is the dynamically computed context vector. The output probability distribution is then:

$$p(y_t | y_1, \dots, y_{t-1}, X) = g(y_{t-1}, s_t, c_t) \quad (9)$$

### 1.1.3 Implementation Details

**Model Architecture:**

- **Embedding Dimension:** 128
- **Hidden Dimension:** 512
- **Number of RNN Layers:** 2
- **Batch Size:** 32
- **Number of Epochs:** 20
- **Learning Rate:**  $1 \times 10^{-4}$
- **Optimizer:** Adam

**Embedding Layer:** An embedding layer transforms discrete token indices into continuous vector representations. Given a vocabulary size  $V$  and embedding dimension  $d_{emb}$ , the embedding matrix  $E \in \mathbb{R}^{V \times d_{emb}}$  is learned during training. For each token index  $x_i$ , the embedding is simply:

$$\text{emb}(x_i) = E[x_i] \quad (10)$$

**Teacher Forcing:** During training, we employ teacher forcing with a ratio of 0.5. This technique involves using the ground-truth token  $y_{t-1}$  instead of the model’s prediction  $\hat{y}_{t-1}$  as input to the decoder at time step  $t$ . This helps stabilize training by providing the model with correct historical context, though we randomly apply this to only 50% of training batches to prevent over-reliance on ground truth during inference.

**Handling Variable-Length Sequences:** Since maze descriptions and paths vary in length, we employ padding to create uniform batch sizes. Sequences are padded to the maximum length in each batch using a special padding token. During loss computation, we apply masking to ensure that padding tokens do not contribute to the gradient updates.

**Loss Function:** We use cross-entropy loss over the output vocabulary at each time step:

$$\mathcal{L} = -\frac{1}{T_y} \sum_{t=1}^{T_y} \log p(y_t^* | y_1^*, \dots, y_{t-1}^*, X) \quad (11)$$

where  $y_t^*$  denotes the ground-truth token at position  $t$ , and padding positions are masked out.

## 1.2 Results and Observations

### 1.2.1 Evaluation Metrics

We evaluate our RNN model using two primary metrics:

1. Sequence Accuracy (Exact Match)
2. F1-score (For partial correctness)

### 1.2.2 Training and Validation Performance

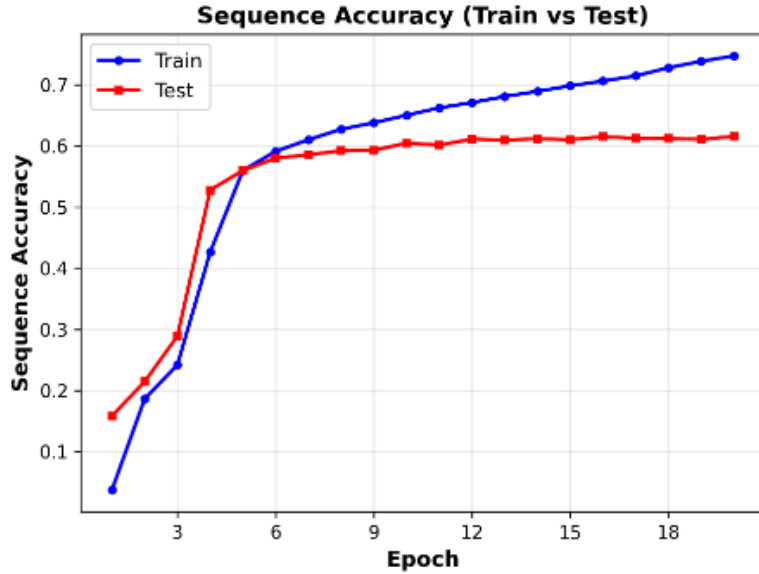


Figure 1: Sequence Accuracy vs Epoch for RNN model across training, validation, and test sets

Figure 1 shows the sequence accuracy across epochs for the training and test sets.

**Observations:**

- **Final Train Sequence Accuracy:** 74.70%
- **Final Test Sequence Accuracy:** 61.55%

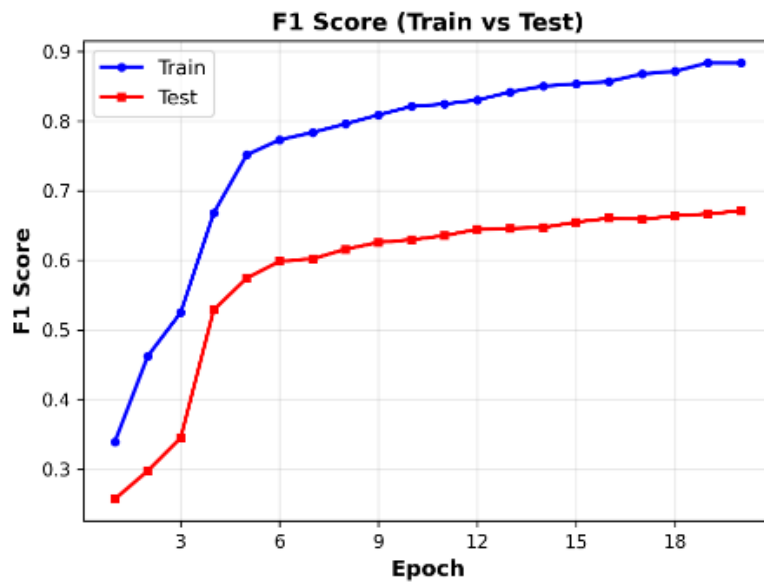


Figure 2: F1-Score vs Epoch for RNN model across training and test sets

Figure 3 presents the F1-scores across epochs.

**Observations:**

- **Final Train F1-score:** 0.8839
- **Final Test F1-score:** 0.6708

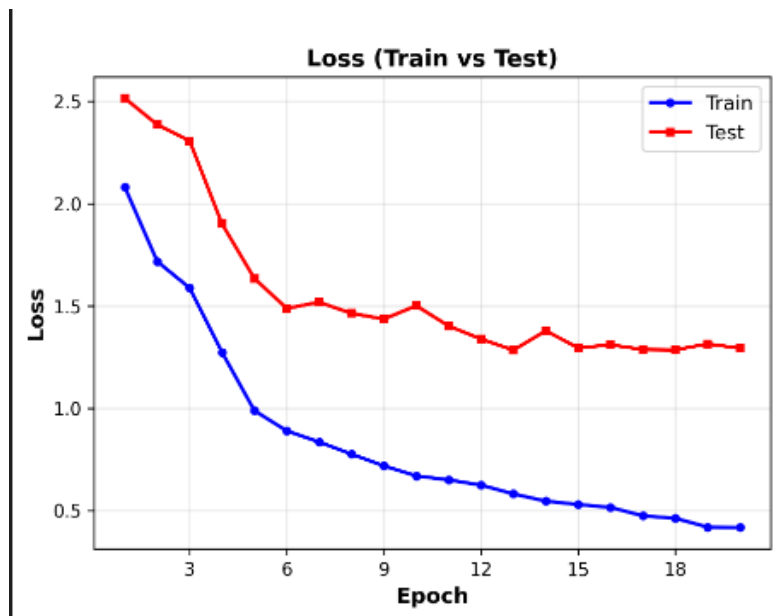


Figure 3: Loss vs Epoch for RNN model across training and test sets

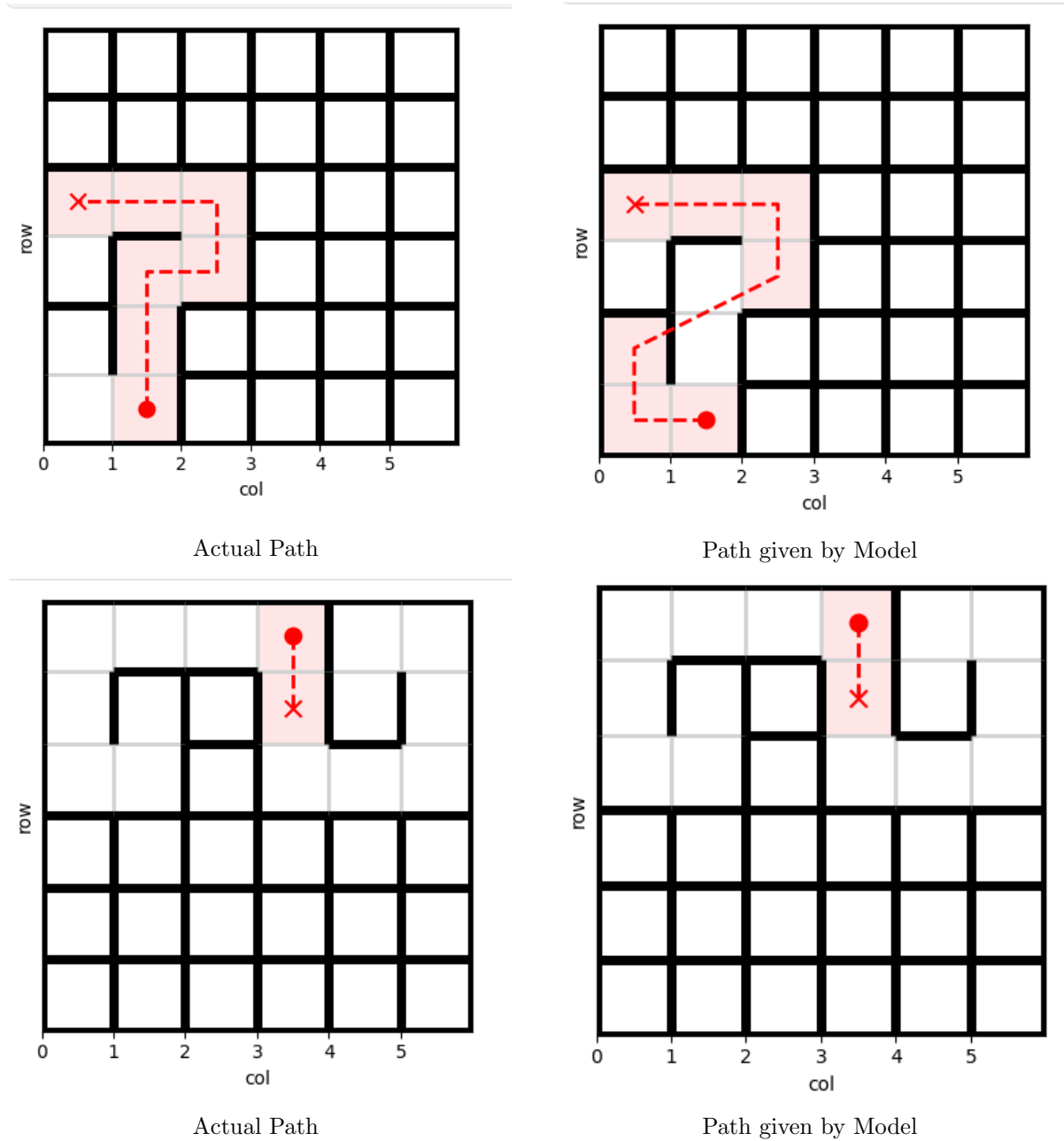
Figure 3 presents the F1-scores across epochs.

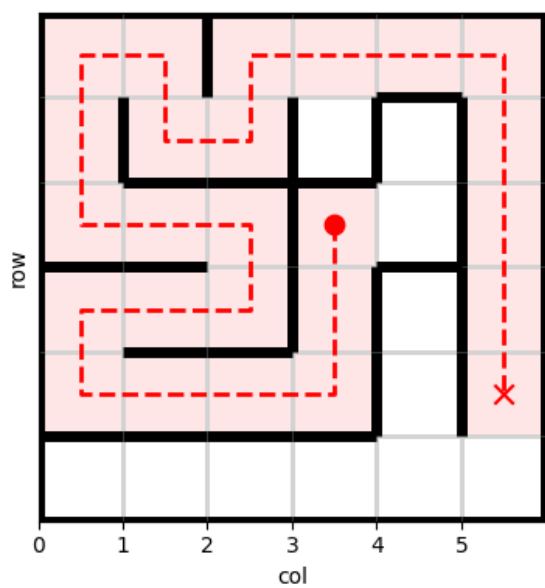
**Observations:**

- **Final Train Loss:** 0.4171
- **Final Test Loss:** 1.2933

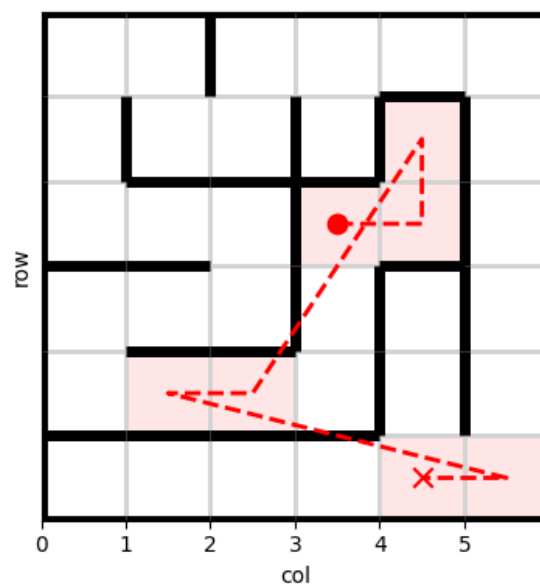
1.2.3 Qualitative Analysis: Maze Visualizations

To better understand the model’s behavior, we visualized predictions on 5 randomly selected mazes from the validation set.

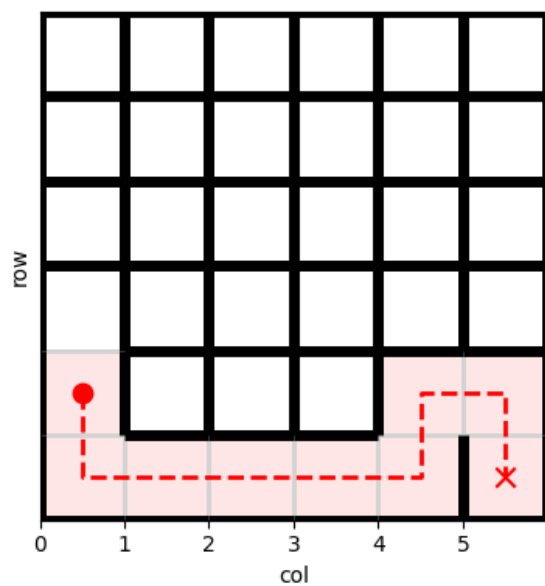




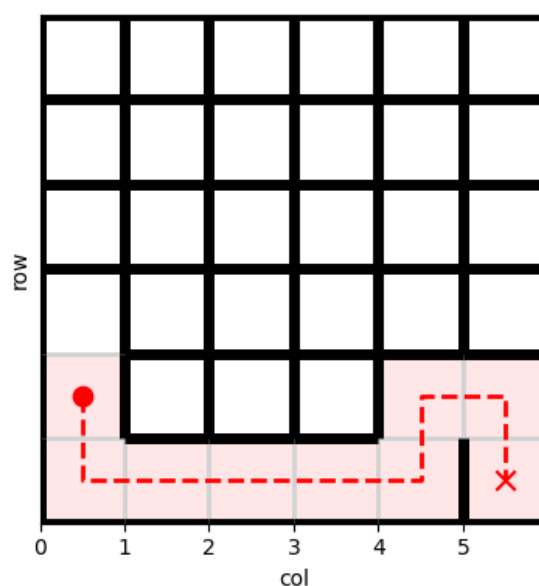
Actual Path



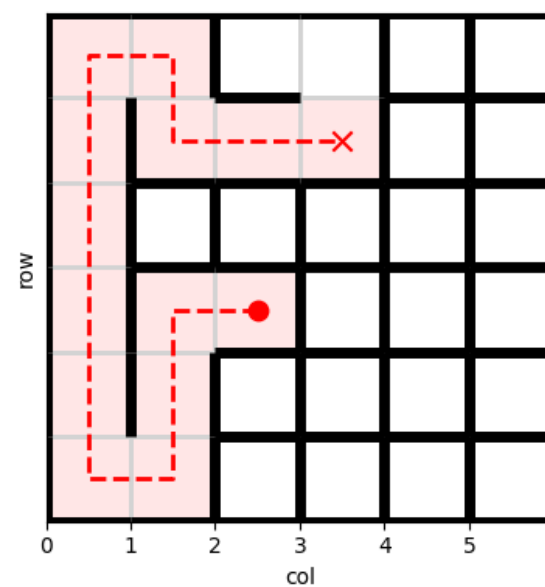
Path given by Model



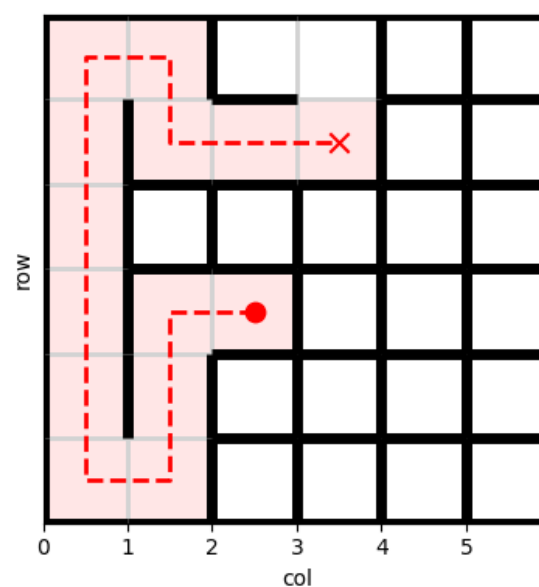
Actual Path



Path given by Model



Actual Path



Path given by Model

Figure 5: Correct paths (Left) vs Predictions made by the model (Right)

## 2 Transformer

### 2.1 Theory of Transformer Architecture

#### 2.1.1 Overview

The Transformer architecture relies entirely on attention mechanisms to draw global dependencies between input and output. Unlike RNNs, which process data sequentially, Transformers process the entire sequence in parallel, allowing for better handling of long-range dependencies—a crucial feature for understanding complex maze structures.

For this task, we implement an Encoder-Decoder Transformer:

**Encoder:** The encoder consists of a stack of  $N = 6$  identical layers. Each layer has two sub-layers: a multi-head self-attention mechanism and a simple, position-wise fully connected feed-forward network. The encoder takes the maze description sequence  $X$  and maps it to a continuous representation  $Z$ . This allows every token in the maze description (e.g., walls, start, end) to attend to every other token, creating a rich, contextualized map of the maze topology.

**Decoder:** The decoder is also a stack of  $N = 6$  identical layers. In addition to the two sub-layers present in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. This cross-attention mechanism allows the decoder to focus on relevant parts of the maze structure (walls or open paths) while generating the next coordinate in the path  $Y$ .

#### 2.1.2 Self-Attention and Cross-Attention

The core component of the Transformer is the Scaled Dot-Product Attention. An attention function maps a query and a set of key-value pairs to an output.

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (12)$$

where:

- **Self-Attention (Encoder):**  $Q, K, V$  all come from the output of the previous encoder layer. This allows positions in the encoder to attend to all positions in the previous layer.
- **Masked Self-Attention (Decoder):**  $Q, K, V$  come from the decoder’s previous layer. Crucially, we apply a causal mask (setting future positions to  $-\infty$  before softmax) to ensure the prediction for position  $i$  can depend only on known outputs at positions less than  $i$ .
- **Cross-Attention:** The queries  $Q$  come from the previous decoder layer, while the keys  $K$  and values  $V$  come from the output of the encoder. This allows every position in the decoder to attend to all positions in the input sequence.

#### 2.1.3 Positional Encoding

Since the Transformer contains no recurrence and no convolution, the model itself has no inherent sense of the order of the sequence. To inject information about the relative or absolute position of the tokens in the sequence, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks.

We use sinusoidal positional encodings as defined in the original paper:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}}) \quad (13)$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}}) \quad (14)$$

where  $pos$  is the position and  $i$  is the dimension.

### 2.1.4 Implementation Details

**Hyperparameters:**

- **Model Dimension ( $d_{model}$ ):** 128
- **Feedforward Dimension ( $d_{ff}$ ):** 512
- **Attention Heads ( $h$ ):** 8
- **Number of Layers ( $N$ ):** 6 (Encoder) + 6 (Decoder)
- **Dropout:** 0.1
- **Batch Size:** 32
- **Optimizer:** Adam with Learning Rate  $1 \times 10^{-4}$

**Training Strategy:** We utilize teacher forcing during training by shifting the target sequence. The input to the decoder is the target sequence shifted right (prepended with `<PATH_START>`), and the target for the loss function is the original sequence. This enforces the auto-regressive property where  $p(y_t|y_{<t}, X)$ .

Rest implementations like teacher forcing, embedding layer are same as in Recurrent neural network part.

## 2.2 Results and Observations

### 2.2.1 Training and Validation Performance

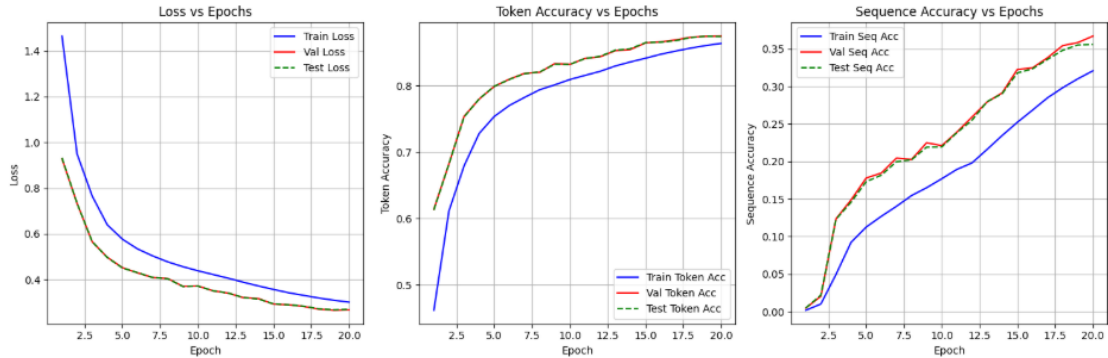


Figure 6: accuracies vs epoches

Figure 6 illustrates the learning progress of the Transformer.

**Observations-comparisions:**

- **The Accuracy Paradox (High F1 vs. Low Sequence Accuracy):** Transformer achieves a much higher F1-score (0.87 vs 0.67) but a significantly lower Sequence Accuracy (35% vs 61%).

- *Interpretation:* This suggests the Transformer correctly predicts the vast majority of the tokens in the path (hence high F1), but frequently makes a single mistake that invalidates the entire sequence (hence low Exact Match).
- *Exposure Bias:* The RNN was trained with a teacher-forcing ratio of 0.5, meaning it learned to recover from its own mistakes during training. The Transformer was trained with 100% teacher forcing (via masking), meaning it never experienced its own errors during training. During inference, a single deviation causes a cascading error that the Transformer cannot recover from, ruining the Sequence Accuracy.
- **Global vs. Local Context:** The Transformer’s Test Loss (0.2696) is drastically lower than the RNN’s (1.2933). This indicates that the Transformer’s self-attention mechanism captures the *global* topology of the maze much better than the RNN. It ”understands” the general route from Start to End effectively, whereas the RNN struggles with the long-range dependencies required to plan the full path.
- **Inductive Bias of Sequentiality:** The maze solving task is inherently sequential (step  $t$  is physically adjacent to step  $t - 1$ ). The RNN has a strong inductive bias for this sequentiality. The Transformer, which processes data in parallel and relies on Positional Encodings, may occasionally predict ”teleportation” (jumping to a non-adjacent cell) if the attention mechanism focuses too heavily on the global goal rather than the immediate local constraint.

### 2.2.2 Comparison with RNN

Comparing the two architectures on the validation set:

| Metric              | RNN (Bahdanau) | Transformer |
|---------------------|----------------|-------------|
| Final Test Loss     | 1.2933         | 0.2696      |
| Final Test F1-Score | 0.6708         | 0.8741      |
| Sequence Accuracy   | 61.55%         | 35.58%      |

Table 1: Performance comparison between RNN and Transformer models.

### 2.2.3 Qualitative Analysis

Below is a visualization of a prediction from the Transformer model on the validation set. The model successfully navigates the maze constraints.

## 2.3 Visualisation of predictions:

Sample 1958:  
 Ground Truth: (1,3) (0,3) (0,2) (0,1) (0,0) (1,0) (2,0) (3,0) (4,0) (5,0) (5,1) (5,2) (5,3) (4,3) (3,3) (2,3) (2,4) (2,5)  
 Predicted: (1,3) (0,3) (0,2) (0,1) (0,0) (1,0) (1,1) (2,1) (2,0) (3,0) (4,0) (5,0) (5,1) (5,2) (5,3) (4,3) (4,4) (5,4) (5,5) (4,5) (3,5) (2,5)  
 Exact Match: False | F1 Score: 0.7500

Prediction Sample 1958 (Exact Match: False)

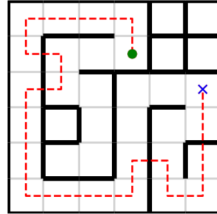


Figure 7: sample-1

Sample 4375:  
 Ground Truth: (3,2) (3,3) (3,4) (3,5) (4,5) (4,4) (5,4)  
 Predicted: (3,2) (3,3) (3,4) (3,5) (4,5) (4,4) (5,4)  
 Exact Match: True | F1 Score: 1.0000

Prediction Sample 4375 (Exact Match: True)

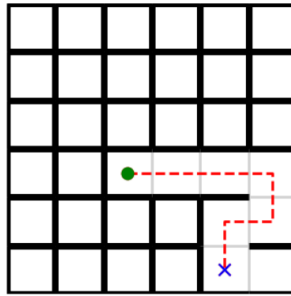


Figure 8: sample-2

Sample 6141:  
 Ground Truth: (5,5) (4,5) (3,5) (2,5) (2,4) (3,4) (3,3) (2,3)  
 Predicted: (5,5) (4,5) (3,5) (2,5) (2,4) (2,3)  
 Exact Match: False | F1 Score: 0.8571

Prediction Sample 6141 (Exact Match: False)

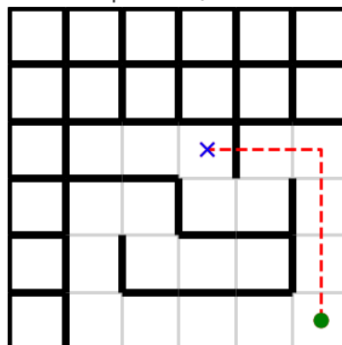


Figure 9: sample-3

Sample 7793:  
 Ground Truth: (4,2) (3,2) (3,3) (2,3)  
 Predicted: (4,2) (4,1) (4,0) (5,0) (5,1) (5,2) (5,3) (5,4) (4,4) (3,4) (2,4) (2,3)  
 Exact Match: False | F1 Score: 0.2500

Prediction Sample 7793 (Exact Match: False)

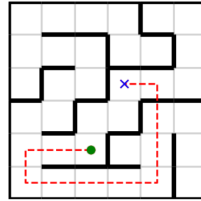


Figure 10: sample-4

Sample 6032:  
 Ground Truth: (3,1) (3,2) (4,2) (4,1) (5,1) (5,0) (4,0) (3,0) (2,0) (2,1)  
 Predicted: (3,1) (2,1)  
 Exact Match: False | F1 Score: 0.3333

Prediction Sample 6032 (Exact Match: False)

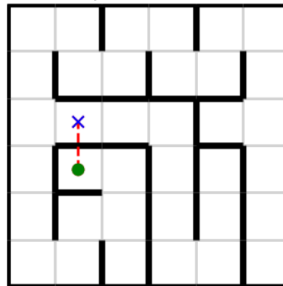


Figure 11: sample-5